# OSSP – PRATICAL

G HANSITA – 2520030332
SEC – 9A

## PRIORITY SCHEDULING

root@Hansi: ~
GNU nano 7.2

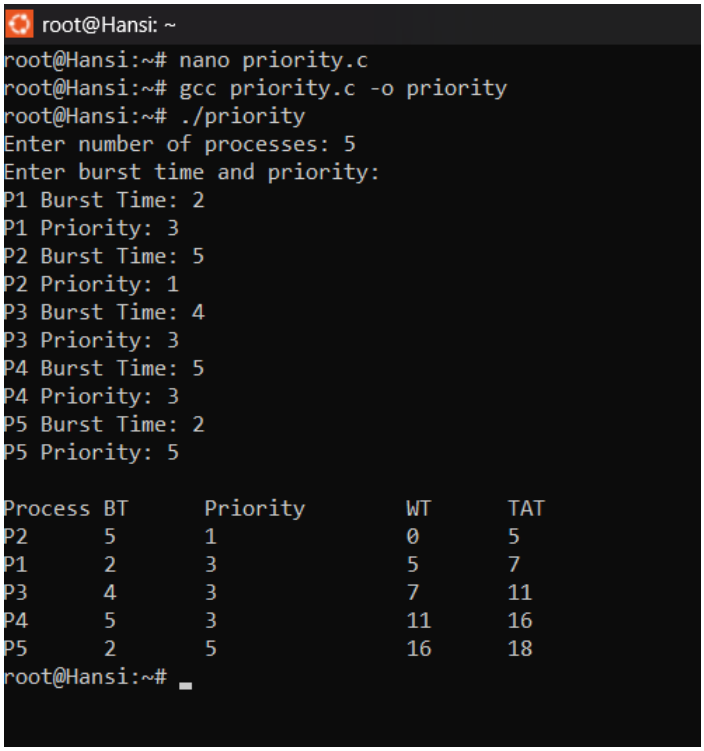
```c
#include <stdio.h>
int main()
{
    int n, i, j;
    int bt[10], priority[10], p[10];
    int wt[10], tat[10];
    int temp;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time and priority:\n");
    for(i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);
        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }
    /* Sort according to priority */
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(priority[i] > priority[j])
            {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

    wt[0] = 0;
    for(i = 1; i < n; i++)
    {
        wt[i] = wt[i - 1] + bt[i - 1];
    }
    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
    }
    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");
    for(i = 0; i < n; i++)
```

```c
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t\t%d\t%d\n",
               p[i], bt[i], priority[i], wt[i], tat[i]);
    }
    return 0;
}
```


root@Hansi: ~
root@Hansi:~# nano priority.c
root@Hansi:~# gcc priority.c -o priority
root@Hansi:~# ./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 3
P2 Burst Time: 5
P2 Priority: 1
P3 Burst Time: 4
P3 Priority: 3
P4 Burst Time: 5
P4 Priority: 3
P5 Burst Time: 2
P5 Priority: 5
Process BT Priority WT TAT
P2 5 1 0 5
P1 2 3 5 7
P3 4 3 7 11
P4 5 3 11 16
P5 2 5 16 18
root@Hansi:~#

## ROUND ROBIN

root@Hansi: ~

GNU nano 7.2

```c
#include <stdio.h>
int main()
{
    int n, i;
    int bt[10], rem[10];
    int tq, time = 0;
    int done;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst time:\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
    }
    printf("Enter time quantum: ");
    scanf("%d", &tq);
    printf("\nExecution:\n");
    while(1)
    {
        done = 1;
        for(i = 0; i < n; i++)
        {
            if(rem[i] > 0)
            {
                done = 0;
                printf("P%d ", i + 1);
                if(rem[i] > tq)
                {
                    time = time + tq;
                    rem[i] = rem[i] - tq;
                }
                else
                {
                    time = time + rem[i];
                    rem[i] = 0;
                }
            }
        }
        if(done == 1)
            break;
    }
    printf("\nTotal time = %d\n", time);
    return 0;
}
```

root@Hansi: ~

```
root@Hansi:~# nano roundrobin.c
root@Hansi:~# gcc roundrobin.c -o roundrobin
root@Hansi:~# ./roundrobin
Enter number of processes: 3
Enter burst time:
P1: 2
P2: 1
P3: 2
Enter time quantum: 1

Execution:
P1 P2 P3 P1 P3
Total time = 5
root@Hansi:~#
```

# IPC USING PIPE

root@Hansi: ~

GNU nano 7.2

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
int main()
{
    int fd[2];
    char write_msg[] = "Hello from Parent";
    char read_msg[100];
    pipe(fd);
    if(fork() == 0)
    {
        close(fd[1]);
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);
    }
    else
    {
        close(fd[0]);
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);
    }
    return 0;
}
```

root@Hansi: ~

```
root@Hansi:~# nano pipe.c
root@Hansi:~# gcc pipe.c -o pipe
root@Hansi:~# ./pipe
Child received: Hello from Parent
root@Hansi:~#
```

## IPC USING SHARED MEMORY

root@Hansi: ~

GNU nano 7.2

```c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
int main()
{
    int shmid;
    char *str;
    shmid = shmget(1234, 1024, 0666 | IPC_CREAT);
    str = (char *)shmat(shmid, NULL, 0);
    printf("Enter message: ");
    fgets(str, 100, stdin);
    printf("Message stored in shared memory: %s", str);
    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

root@Hansi: ~

```
root@Hansi:~# nano shared_memory.c
root@Hansi:~# gcc shared_memory.c -o shared_memory
root@Hansi:~# ./shared_memory
Enter message: Hello from Shared Memory
Message stored in shared memory: Hello from Shared Memory
root@Hansi:~#
```